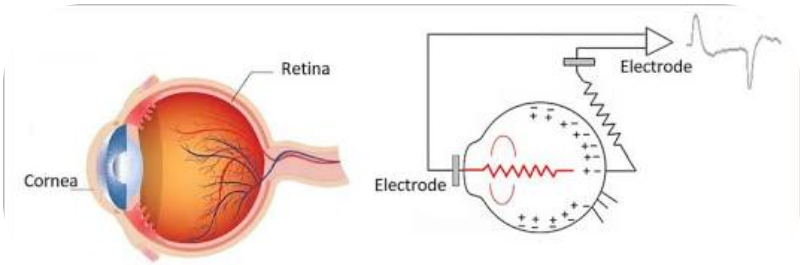
what is eog signals what waves they have and how does the EOG capture work can it be use for continuous eye tracking?

EOG (Electrooculography) is a technique used to measure the electrical potential difference between the front and the back of the human eye.

1. What are EOG Signals?

The human eye acts like a tiny battery or a dipole. The **cornea** (the front) holds a positive charge, while the **retina** (the back) holds a negative charge. This constant electrical field is called the **corneo-retinal potential**.

When you move your eyes, this dipole shifts relative to the skin around your eye socket. EOG signals are the changes in electrical voltage recorded from the skin as the eye moves.



The Corneo-Retinal Dipole Principle. Source: ResearchGate

2. How Does EOG Capture Work?

EOG captures eye movements using pairs of small electrodes placed on the skin around the eyes:

Horizontal Movements: One electrode is placed on the outer corner (canthus) of the left eye, and another on the outer corner of the right eye.

Vertical Movements: One electrode is placed above the eyebrow and another just below the eye.

Ground/Reference: A final electrode is placed elsewhere (like the forehead or earlobe) to filter out background electrical noise.

When you look to the right, the positively charged cornea moves closer to the right electrode, causing the recorded voltage to jump positive. When you look left, the voltage drops negative. An amplifier and a micro-controller process these tiny voltage changes (usually between 15 to 200 microvolts) into usable digital data.

3. What Waves/Signal Features Do They Have?

Unlike an EEG (brain waves) or an ECG (heart rhythms), EOG signals don't look like rhythmic sine waves. Instead, EOG signals track distinct **voltage shifts** corresponding to specific types of eye movements:

Saccades (Rapid Shifts): Sharp, abrupt step-like wave peaks. They show up when your eyes quickly jump from one point to another (e.g., reading a line of text).

Smooth Pursuit (Tracking): Slower, ramp-like linear waves. These appear when your eyes smoothly track a moving object.

Blinks: Very sharp, high-amplitude spikes that primarily appear on the vertical channel.

Fixations: Flat, stable baseline plateaus where the voltage remains steady because the eye is holding still.

4. Can EOG Be Used for Continuous Eye Tracking?

Yes, but it has major technical limitations compared to modern camera-based (optical) eye trackers.

While EOG is great for certain applications, it struggles as a standalone *continuous, high-precision* eye tracker due to specific trade-offs:

The Advantages

Low Computational Power: Processing a few voltage streams requires very little battery and CPU power compared to processing high-definition video frames.

Works in Darkness / Eyes Closed: Because it measures internal electrical fields, EOG can track eye movements in total darkness or even when your eyes are closed (making it excellent for sleep studies/REM tracking).

Form Factor: It can easily be integrated into the frames of smart glasses or VR headsets using subtle, skin-contact electrodes.

The Major Limitations (Why it isn't perfect for continuous tracking)

Signal Drift: This is the biggest hurdle. The baseline corneo-retinal potential changes dynamically based on metabolic factors, muscle fatigue, alertness, and—most frustratingly—**light adaptation** (your eye's electrical charge shifts significantly when moving from a dark room to a bright room). This means the system requires constant recalibration.

Muscle Artifacts: Frowning, chewing, or blinking injects massive electrical noise (EMG signals) into the EOG reading, which can easily confuse the tracking algorithm.

Poorer Angular Precision: EOG can easily tell *which direction* you looked, but it struggles to pinpoint the *exact coordinate* on a screen you are staring at compared to camera-based trackers.

Current Use Case: Today, EOG is rarely used to continuously point-and-click on screens. Instead, it is highly successful in **human-computer interfaces (HCI)** for users with paralysis (using intentional blinks or hard left/right darting movements to trigger actions) and in sleep research.

So I have a GUI which we can use for continuous tracking such as:
1.I have a top view(Top view X,Y plane)GUI 2D screen of a ball in a square grid which can controlled by the vertical and horizontal eye movement.
2.Side view(STANDING X,Y)2D view of the plane with the ball on the plane which will move accordingly to the vertical and horizontal eye movement

That is a classic, smart way to map 2D signals into multi-dimensional control. Because EOG naturally splits into independent horizontal (X) and vertical (Y) voltage channels, it matches your dual-view setup perfectly.

To make this system work reliably for continuous control without the ball drifting off-screen or jittering uncontrollably, you will need to map your signal logic carefully.

1. How the Mapping Works

Because you have a **Top View** and a **Side View**, you are essentially mapping your two physiological voltage signals into two separate 2D coordinate spaces:

View	GUI Axis	EOG Source Channel	Eye Moven
Top View (X, Y)	**Horizontal (X)**	Horizontal Channel	Looking Le
	Vertical (Y)	Vertical Channel	Looking Up
Side View (X, Z)	**Horizontal (X)**	Horizontal Channel	Looking Le
	Vertical (Z)	Vertical Channel	Looking Up

In this configuration, your two views are perfectly synchronized because they share the exact same raw data stream.

2. Design Strategy: Absolute vs. Relative Control

When building the backend loop for this GUI, you must choose how the eye movement actually translates to ball movement.

Because of the **signal drift** inherent to EOG, **Relative Control (Velocity Mapping)** is significantly more reliable than Absolute Control (Position Mapping).

Method A: Relative Control (Highly Recommended)

Instead of forcing the ball to sit exactly where your eye is looking, use your eye position as a **joystick**.

> **The Logic:** Looking slightly right moves the ball to the right at a slow speed. Looking *hard* right moves it faster. Returning your eyes to a neutral, forward fixation stops the ball where it is.
>
> **Why it works for EOG:** You don't need a perfectly calibrated absolute voltage map. As long as the system knows your eyes crossed a certain threshold to the left or right, it moves the ball smoothly.

Method B: Absolute Control

The ball mimics your exact eye coordinates. If you look at the top-right corner of the grid, the ball teleports to the top-right corner.

> **Why it fails with EOG:** The moment your skin sweats slightly or the room's lighting changes, the "center" point will drift. The ball will start slowly crawling toward an edge even though you are looking straight ahead.

3. Critical Signal Processing Steps for Your GUI

To prevent your ball from jittering or misbehaving on screen, your input pipeline should process the raw voltage through three stages before updating the GUI frames:

```
otch Filter] ──> [Dead-Zone Filter] ──> [Blink Blanking] ──> [GUI Ball Movement]
```

1. **The Dead-Zone Filter:** Human eyes naturally micro-saccade (make tiny involuntary jumps even when staring at one spot). Implement a "dead-zone" in your code. If the voltage change is less than a specific threshold (e.g., $\pm 15 \mu V$), treat it as zero so the ball stays perfectly still.
2. **Blink Blanking (The Vertical Channel Hack):** Every time your user blinks, the vertical channel will see a massive positive spike. If unmanaged, **the ball will aggressively shoot upward every time the user blinks**. Your code must detect these high-velocity spikes and temporarily "freeze" the Y/Z position of the ball for roughly 200–300 milliseconds until the blink passes.
3. **Active Baseline Reset:** Give your user a physical button (like a spacebar) or an intentional double-blink command. When triggered, the software snaps the current EOG voltage straight to $(0, 0)$ on the grid, instantly recentering the tracking when drift inevitably sets in.

For this hardware I'm using is ADS1115 and Arduino UNO ,EXG Bioamp pill,

That hardware combination is a highly effective setup for this project. The **BioAmp EXG Pill** does the heavy lifting by filtering and amplifying the raw bio-potentials, while the 16-bit **ADS1115** offers the resolution needed to capture fine analog voltage shifts that the standard 10-bit Arduino UNO pins would completely miss.

*Note: Since you want both horizontal and vertical tracking, you will need **two separate BioAmp EXG Pills** (one for the X-axis and one for the Y-axis).*

1. The Hardware Wiring Configuration

The ADS1115 communicates over I^2C. You will power both the EXG pills and the ADC module from the Arduino's **5V** line to maximize signal swing range.

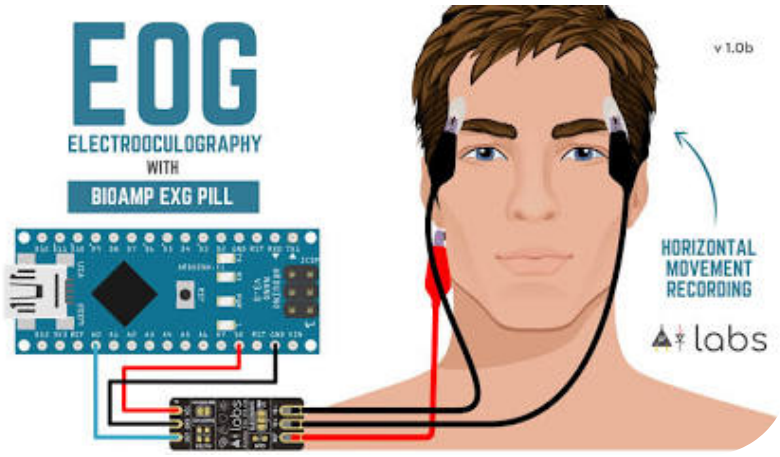
BioAmp Pill 1 (Horizontal)	ADS1115 Pin	Arduino UNO Pin
VCC	—	**5V**
GND	—	**GND**
OUT	**A0**	—

BioAmp Pill 2 (Vertical)	ADS1115 Pin	Arduino UNO Pin
VCC	—	**5V**
GND	—	**GND**
OUT	**A1**	—

ADS1115 Pin	Arduino UNO Pin
VDD	**5V**
GND	**GND**
SCL	**A5** (or dedicated SCL pin)
SDA	**A4** (or dedicated SDA pin)

2. Electrode Placement Using Two EXG Pills

To cleanly separate your axes, arrange your gel electrodes exactly like this:



Horizontal EOG Electrode Layout. Source: Crowd Supply

Pill 1 (Horizontal): Place the positive (+) electrode on the right temple, the negative (-) electrode on the left temple.

Pill 2 (Vertical): Place the positive (+) electrode above your right eyebrow, the negative (-) electrode right below your right eye.

Reference / Ground: Connect the reference cable (REF) from *both* pills to a single neutral spot, like the bony bump right behind your ear (mastoid process).

3. Arduino Code Framework (Joystick/Relative Logic)

This code reads the data from the ADS1115, implements a digital dead-zone, and outputs relative adjustment steps over the serial port. You can feed this data stream directly into your GUI environment (such as Python/Pygame, Processing, or Unity).

Make sure to install the **Adafruit_ADS1X15** library in your Arduino IDE first.

C++

```cpp
#include <Wire.h>
#include <Adafruit_ADS1X15.h>

Adafruit_ADS1115 ads;

// Baseline values determined during startup calibration
int16_t baselineX = 0;
int16_t baselineY = 0;

// Fine-tune these parameters based on your testing
const int16_t DEADZONE = 400;     // Ignores micro-jitters
const float SENSITIVITY = 0.005;   // Multiplier to scale speed of the ball

void setup(void) {
  Serial.begin(115200);

  if (!ads.begin()) {
    Serial.println("Failed to initialize ADS1115!");
    while (1);
  }

  // Set gain to GAIN_ONE (+/- 4.096V range) to match BioAmp output swing
  ads.setGain(GAIN_ONE);

  // Perform a 2-second initial calibration. User must look straight ahead!
  long sumX = 0;
  long sumY = 0;
  int samples = 200;

  for(int i = 0; i < samples; i++) {
    sumX += ads.readADC_SingleEnded(0);
    sumY += ads.readADC_SingleEnded(1);
    delay(10);
  }
  baselineX = sumX / samples;
  baselineY = sumY / samples;
```

```
}

void loop(void) {
  int16_t rawX = ads.readADC_SingleEnded(0);
  int16_t rawY = ads.readADC_SingleEnded(1);

  // Calculate deviation from baseline center
  int16_t deltaX = rawX - baselineX;
  int16_t deltaY = rawY - baselineY;

  // Relative movement speed values
  float moveX = 0;
  float moveY = 0;

  // Apply horizontal Deadzone
  if (abs(deltaX) > DEADZONE) {
    moveX = deltaX * SENSITIVITY;
  }

  // Apply vertical Deadzone + Basic Blink Mitigation
  // (Blinks cause massive positive spikes. If spike is too high, ignore it)
  if (abs(deltaY) > DEADZONE && abs(deltaY) < 8000) {
    moveY = deltaY * SENSITIVITY;
  }

  // Print relative vectors for the GUI to consume
  // Format: X_speed, Y_speed
  Serial.print(moveX);
  Serial.print(",");
  Serial.println(moveY);

  delay(20); // ~50Hz sampling rate matches GUI frame updates well
}
```

4. Handling Your Two GUI Views

Your computer-side script (e.g., Python) reads these incoming values every frame and updates the positions coordinates continuously:

1. **Top View Grid:** `ballX += moveX` and `ballY += moveY`
2. **Side View Grid:** `ballX += moveX` and `ballZ += moveY` (mapping the vertical